

CSIS 2300 Introduction to Algorithms

Spring 2026 – Homework 10 Solutions

(1) A diplomatic conference is having a large banquet. To promote diplomatic relations, the organizers wish to seat people so that no two people from the same country share a table.

There are n countries represented at the conference. Countries have sent different-sized delegations; let p_i be the number of people present from country i . The banquet hall has m tables in it of various sizes; let s_j be the number of seats at table j .

Show how to find a valid seating arrangement (or prove that none exists) by modeling the situation as a network flow problem. Be sure to explain: (a) what all of the nodes in your flow network represent, (b) what each edge represents, (c) what the capacity of each edge should be, and (d) how to obtain an actual seating chart from the network flow.

As with most such problems, the most intuitive approach is a layer of nodes for each type of entity you wish to model: people, countries, and tables.

The first layer has $\sum_i p_i$ nodes – one for every person. The source connects to each of these with a capacity-1 edge. This guarantees that no person is assigned to sit in more than one spot.

The second layer has n nodes, one per country. Each person connects to their country's node with a capacity-1 edge. (*Note: These first two layers can easily be combined. Just have a country layer, and the source connects to each country with capacity p_i .*)

The third layer has m nodes, one per table. Every country connects to every table with a capacity-1 edge, which guarantees no country takes more than one seat at any table.

Finally, the table nodes connect to the sink with edges of capacity s_j ; this guarantees that we will not overfill any table.

Any valid flow on this network thus corresponds to a valid seating of a number of people equal to the flow volume; further, every legal seating arrangement corresponds to a valid flow. The maximum possible flow on this network is $\sum_i p_i$ (the total capacity leaving the source. If MaxFlow finds a flow of this volume, then we have seated everyone; if it does not, no valid arrangement for all guests is possible.

(2) You are given a flow network G and its corresponding maximum flow f . (i.e. Someone has already run Ford-Fulkerson and is telling you the answer it returned.)

Now, one particular edge $e \in G$ has its capacity increased by one unit. We want to find the maximum flow f' in this (slightly) modified graph G' .

(a) Prove that either $vol(f') = vol(f)$ or $vol(f') = vol(f) + 1$.

(b) Give a process for finding the new maximum flow in $O(V + E)$ time. (Note that this is not enough time to run F-F again.)

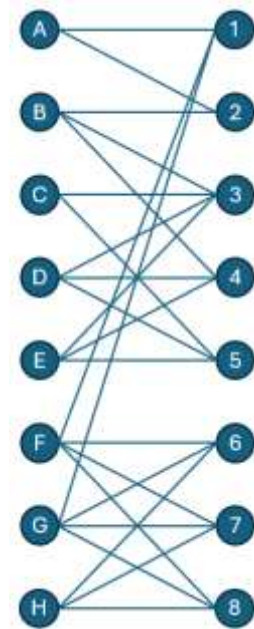
(a) We know that max flow is equal to the minimum capacity cut. If e does not cross the mincut for f (or if there are other equivalent mincuts that do not involve e), then the minimum cut for f' is unchanged, and thus $vol(f') = vol(f)$. If e does cross every mincut for f , then the capacity of this cut has increased by 1. Because all capacities are integers, every other cut must have had capacity of at least $vol(f) + 1$, and so the new cut involving e is a minimum for f' , and $vol(f') = vol(f) + 1$.

(b) From f , we can construct the corresponding residual graph in $O(V + E)$ time. Now, we need to run only a single pass of Ford-Fulkerson – either we will find no augmenting path, or any augmenting path we find will increase the flow by exactly 1 (see part (a)). This single pass also takes $O(V + E)$ time.

(3) Consider the following greedy algorithm for finding a maximum bipartite matching: Begin with an empty matching M . Sort the vertices by degree (the number of nodes they are connected to) and then process them in ascending order. Each time we find an unmatched vertex u , match it with its unmatched neighbor v of lowest degree. If u has no unmatched neighbors, just skip it.

Either prove that this algorithm always finds the maximum bipartite matching, or give a counterexample that shows that it does not.

This one proved more challenging than I was expecting – the smallest counterexample I could find (and I suspect the smallest possible) has eight nodes per side, as shown at right. This greedy algorithm always produces a maximal matching (one that cannot be improved by adding edges), but not necessarily the maximum matching.



Note in the pictured graph that a perfect matching is possible by simply pairing the nodes horizontally (A-1, B-2, etc.) However, the stated greedy algorithm will initially match A-2 (since those are the only two degree-2 nodes in the graph).

From there, we're already doomed. The four nodes B-E only have three possible partners (3-5), so one of them will remain unpaired, and we will not find the maximum eight pairs.

(4) A variant of the stable matching problem is the *roommate problem*, in which there is only one group of entities (as opposed to applicants/employers in the base problem). We assume an even number of people that are going to be paired up as roommates (a perfect matching). Each person in the group has a ranking of all of the other individuals in the group. The matching is stable in this case if there is no pair of non-roommates who prefer each other to their actual roommates.

Give an instance of this problem (i.e. a list of people and their rankings) that does not have a stable matching. Be sure to explain how you know there is no stable matching.

Even with four people will do the trick. The preferences are:

A: D,C,B

B: D,A,C

C: A,D,B

D: C,A,B

There are only three possible pairings:

A-B & C-D: Here, A & C would rather pair with each other.

A-C & B-D: Now, A & D would rather pair with each other.

A-D & B-C: Finally, C & D would rather pair with each other.

Thus, there are no stable pairings for this preference set.

(5) A certain string-processing language offers a primitive operation which splits a string into two pieces. Since this operation involves copying the original string, it takes n units of time for a string of length n , regardless of the location of the cut. Suppose, now, that you want to break a string into many pieces. The order in which the breaks are made can affect the total running time. For example, if you want to cut a 20-character string at positions 3 and 10, then ordering the cuts as [3,10] incurs a total cost of $20 + 17 = 37$, while using the order [10,3] has a better cost of $20 + 10 = 30$.

Give a dynamic programming algorithm that, given the locations of m cuts in a string of length n , finds the minimum cost of breaking the string into $m + 1$ pieces. As with all dynamic programming problems, state clearly the recursive substructure, what $OPT(i)$ will actually mean, and the recurrence relation and only then the pseudocode.

This will end up being one of those problems with a triangular DP table. Let the list of cut locations in ascending order be $c_1, c_2, \dots, c_m$. (In the small example in the problem, $c_1 = 3$ and $c_2 = 10$.) Also define $c_0 = 0$ and $c_{m+1} = n$. We can then set up $OPT(i, j)$ to represent the minimum cost of dividing up the substring starting from position c_i and ending at position c_j . The overall problem is to find $OPT(0, m + 1)$.

This structure gives the recurrence: $OPT(i, j) = (c_j - c_i) + \min_{i < k < j} (OPT(i, k) + OPT(k, j))$. $c_j - c_i$ gives us the cost of processing *this* string, and the two recursive calls to OPT are the costs of processing the two smaller strings. We don't know which two smaller strings, and so we look at all of them. The base cases are $OPT(i, i + 1) = 0$ for all $0 \leq i \leq m$ – when there are no cuts to be made, no cost is incurred.

We must fill out the DP table in diagonal order, starting with $OPT(i, i + 2)$ then $OPT(i, i + 3)$ and so on until we get to the corner cell. The overall runtime will be cubic in m – there are a quadratic number of table entries to fill in, and each requires a linear-time minimization.

```
def string_cut(c,m): # c is an array as defined above
    Declare OPT array of size (m+2) x (m+2)
    Set OPT[i][i+1] = 0 for all 0 <= i <= m
    for x = 2 to (m+1): # x is the "gap" or number of segments
        for i = 0 to (m+1)-x:
            best = ∞
            for k = (i+1) to (i+1)+x-1:
                sum = OPT[i][k] + OPT[k][i+x]
                if sum < best: Set best = sum
            OPT[i][i+x] = best + (c[i+x] - c[i])
    return OPT[0][m+1]
```